



1.

الف- شبه کدی بنویسید که یک عدد طبیعی را دریافت کند، در صورتی که این عدد مربع کامل است مقدار True و در غیر این صورت مقدار False برگرداند.

```
input n
initialize i to 1
while i * i <= n do
    if i * i == n then
        return True
    endif
    i = i + 1
endwhile
return False
```

ب- تابع زبان C زیر را طوری کامل کنید که یک عدد طبیعی به عنوان آرگومان بگیرد، در صورتی که این عدد مربع کامل است مقدار 1 و در غیر این صورت مقدار صفر را برگرداند.

پاسخ:

```
int IsPerfectSquare(int n) {
    for(int i = 1; i*i <= n; i++){
        if(i*i == n){
            return 1;
        }
    }
    return 0;
}
```

پ-یک لیست پیوندی داریم که هر گره شامل اطلاعات زیر است:

```
typedef struct Node Node;
struct Node {
    int data;
    Node* next;
};
```

تابع زیر را طوری تکمیل کنید که سر یک لیست پیوندی از این نوع گره را بگیرد و تمام گره هایی که اعداد موجود در آنها مربع کامل است را حذف کند. دقت کنید که گره اول (head) از نوع dummy است و حاوی اطلاعات نیست.

```
#include <stdio.h> // NULL is defined in 'stdio.h' lib
void deletePerfectSquare(Node* head) {
    ...
}
```

مثال)

input) head -> 6 -> 2 -> 1 -> 9 -> 11 -> 7 -> 4

output) head -> 6 -> 2 -> 11 -> 7

پاسخ:

```
void deletePerfectSquare(Node* head) {
    if (head == NULL)
        return;

    Node* prev = head;          // head is dummy
    Node* current = head->next; // Start from the first actual data node
    while (current != NULL) {
        if (IsPerfectSquare(current->data)) {
            Node* to_delete = current;
            prev->next = current->next; // Bypass the node to be deleted
            current = current->next;
            free(to_delete);           // Free memory
        }
        else {
            prev = current;           // Move both pointers forward
            current = current->next;
        }
    }
}
```

2. یک لیست پیوندی داریم که هر گره آن مقدار هر گره آن صفر یا 1 است و به شکل زیر تعریف می شود:

```
typedef struct Node Node;  
struct Node {  
    int data;  
    Node* next;  
};
```

به دنبال این هستیم که هیچ دو عدد 1 کنار هم قرار نگیرند، همچنین سر و ته این لیست پیوندی نباید عدد 1 باشد. تابع زیر را به گونه ای تکمیل کنید که سر یک لیست پیوندی از را بگیرد و بین هر دو گره که شامل اعداد 1 هستند، یک گره با عدد صفر اضافه کند؛ همچنین باید در صورتی که سر یا ته این لیست پیوندی شامل اعداد 1 هستند نیز یک گره با مقدار صفر اضافه کند. (head فاقد اطلاعات است و از نوع dummy می باشد)

```
#include <stdio.h> // NULL is defined in 'stdio.h' lib  
void insertZero(Node* head) {  
    ...  
}
```

مثال)

input) head -> 0 -> 1 -> 1 -> 0 -> 0 -> 1 -> 1 -> 1 -> 0 -> 1

output) head -> 0 -> 1 -> 0 -> 1 -> 0 -> 0 -> 1 -> 0 -> 1 -> 0 -> 1 -> 0 -> 1 -> 0

پاسخ:

```
#include <stdio.h>  
void insertZero(Node* head) {  
    if (head == NULL)  
        return;  
  
    // Handle the Head: If the first data node is 1, insert a 0 before it  
    if (head->next != NULL && head->next->data == 1) {  
        Node* newNode = (Node*)malloc(sizeof(Node));  
        newNode->data = 0;  
        newNode->next = head->next;  
        head->next = newNode;  
    }  
  
    Node* current = head->next; // Start from the first actual data node
```

```

// Handle the Middle: Insert 0 between consecutive 1s
while (current != NULL && current->next != NULL) {
    if (current->data == 1 && current->next->data == 1) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->data = 0;
        newNode->next = current->next;
        current->next = newNode;
    }
    current = current->next;
}

// Handle the Tail: After the loop, 'current' is the last node(tail)
if (current != NULL && current->data == 1) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = 0;
    newNode->next = NULL;
    current->next = newNode;
}
}

```

3. نوعی لیست پیوندی وجود دارد که هر گره، علاوه بر آدرس گره بعدی، آدرس گره قبلی را نیز نگه میدارد. در این سوال از این نوع لیست پیوندی استفاده میکنیم و هر گره شامل اطلاعات زیر است:

```

typedef struct Node Node;
struct Node {
    int data;
    Node* next;
    Node* prev;
};

```

تابع زیر را طوری کامل کنید که سر و ته یک لیست پیوندی دو طرفه که هر دو dummy هستند (یعنی گره اول و گره آخر حاوی اطلاعات نیستند) را بگیرد و چک کند که آیا این لیست پیوندی متقارن است یا نه. در صورتی که این لیست پیوندی متقارن بود مقدار 1 و در غیر این صورت مقدار صفر را برگرداند

```

int isPalindromeList(Node* head, Node* tail) {
    ...
}

```

(مثال)

input) head <-> 5 <-> 2 <-> 6 <-> 1 <-> tail	output) 0
input) head <-> 2 <-> 7 <-> 7 <-> 2 <-> tail	output) 1
input) head <-> tail(empty list)	output) 1

پاسخ:

```
int isPalindromelist(Node* head, Node* tail) {
    if(head->next == tail){ // the list is empty
        return 1;
    }

    Node* first_node = head;
    Node* last_node = tail;

    while(first_node != last_node){
        if(first_node->data != last_node->data){
            return 0;
        }

        if(first_node->next == last_node){
            break;
        }

        first_node = first_node->next;
        last_node = last_node->prev;
    }
    return 1;
}
```

4. نادر و قادر که برادر های دوقلو هستند، در دانشگاه تهران تحصیل میکنند، به دلیل مشکلات حمل و نقلشان برای رفتن به دانشگاه، آنها میخواهند بدانند که این ترم، چند استاد مشترک دارند! گره های زیر را برای دانشجو و استاد تعریف میکنیم. گره استاد، علاوه بر اطلاعات معمولی، یک اشاره گر به سر یک لیست پیوندی از دانشجویانی که این ترم با آن استاد کلاس دارند را نیز دارد.

```
typedef struct Student Student;
struct Student {
    int id;
    // some fields
    Student* next;
};
```

```
typedef struct Professor Professor;
struct Professor {
    // some fields
    Student* stds;
    Professor* next;
};
```

تابع زیر را طوری کامل کنید که **head** یک لیست پیوندی از استاد ها را به همراه شماره دانشجویی 2 نفر از دانشجویان را بگیرد و تعداد استاد هایی که بین این 2 دانشجو مشترک هستند را برگرداند.

```
int findCommonProf(Professor* head, int first_student, int second_student){
    ...
}
```

پاسخ:

```
int findCommonProf(Professor* head, int first_student, int second_student){
    int common_count = 0;
    Professor* current_professor = head;

    while(current_professor != NULL){
        Student *current_student = current_professor->stds;
        int found_first = 0;
        int found_second = 0;

        while(current_student != NULL){
            if(current_student->id == first_student){
                found_first = 1;
            }
            else if(current_student->id == second_student){
                found_second = 1;
            }
            if(found_first && found_second){
                break;
            }
            current_student = current_student->next;
        }

        if(found_first && found_second){
            common_count++;
        }
        current_professor = current_professor->next;
    }
    return common_count;
}
```

5. در این سوال، اطلاعات هر گره از یک لیست پیوندی به این صورت است:

```
typedef struct Node Node;
struct Node {
    // some fields
    Node* next;
};
```

یک لیست پیوندی داریم که اولین گره آن یک Dummy Node (گره بدون دیتا) است. تابعی بنویسید که سر این لیست (head) و یک عدد صحیح غیرمنفی (shift_size) را دریافت کرده و گره‌های حاوی دیتا را به اندازه آن عدد به سمت چپ بچرخاند. (با توجه به اینکه head از نوع dummy است بنابراین نباید جابجا شود و همواره باید ابتدای لیست باقی بماند. همچنین تضمین می شود shift_size همواره کوچکتر از طول لیست پیوندی است).

```
void shiftLeft(Node* head, int shift_size) {
    ...
}
```

پاسخ:

```
void shiftLeft(Node* head, int shift_size) {
    // If list is empty or shift_size is 0, no action needed
    if (head == NULL || head->next == NULL || shift_size <= 0)
        return;
    // Find the current first and last data nodes
    Node* first_data = head->next;
    Node* last_node = first_data;
    while (last_node->next != NULL) {
        last_node = last_node->next;
    }

    // new_tail is the node at 'shift_size' from the start
    Node* new_tail = first_data;
    for (int i = 0; i < shift_size - 1; i++) {
        new_tail = new_tail->next;
    }
    // Connect the old last node to the old first node to close the gap
    last_node->next = first_data;
    // Set the dummy head to point to the new first node
    head->next = new_tail->next;

    // new_tail must point to NULL
    new_tail->next = NULL;
}
```